UNIVERSITY OF DHAKA
INSTITUTE OF INFORMATION TECHNOLOGY

CSE 201 - Data Structure and Algorithm Lab Final

Comprehensive Solved Past Exam & Scenario-Based Practice Question Sets

PART 1: SOLUTIONS TO SOLVED LAB FINAL EXAM

Question 1: Campus Navigation & Path Discovery

> **Scenario & Description:**
>
> Bulbuli is a new student at IIT. During lunch break, she wants to explore path options from 'IIT' to four different campus cafés:
> 1. Science Café (located at Curzon Hall)
> 2. FBS Food Court (located at FBS)
> 3. DACSU Café (located at Social Science Building)
> 4. TSC Café (located at TSC)
>
> Available Connections: IIT-Pharmacy, Pharmacy-Doyel Chattor, Doyel Chattor-DMC, DMC-TSC, Doyel Chattor-TSC, TSC-Nilkhet, TSC-Jagannath Hall, Jagannath Hall-Fuller Road, Fuller Road-Social Science Building, Doyel Chattor-Curzon Hall, Nilkhet-FBS.
> Task: Write a program to display all simple paths from IIT to each café, comma-separated.

Solution Strategy: Represent the campus map as an undirected Graph using an Adjacency List. Use Depth-First Search (DFS) with backtracking to traverse and record every simple path from the source ('IIT') to each designated target building.

Question 2: Infix Expression Evaluator using Stacks

> **Scenario & Description:**
>
> Write a C++ program that receives an arithmetic expression as a string containing integers, basic arithmetic operators (+, -, *, /), and parentheses, and evaluates its mathematical result.
> Example Input: 2 + 3 + (5 + 7) / 2 + 10
> Expected Output: 21

Solution Strategy: Use the Two-Stack Algorithm (Shunting-Yard inspired). Maintain an Operand Stack for numeric values and an Operator Stack for operators and parentheses, processing operations according to operator precedence rules.

PART 2: SCENARIO-BASED PRACTICE QUESTION SETS

PRACTICE EXAM SET 1: THE SMART CENTRAL LIBRARY SYSTEM

Topics: Sorting (Bubble, Insertion, Selection, Merge, Quick), Searching (Linear, Binary) **Time: 1.5 Hours | Marks: 20**

Question 1 (10 Marks): Hybrid Sorting for Cataloging

Scenario & Requirements:

The Central Library is digitalizing thousands of incoming books. Each book has a unique accession ID, a title, and an arrival timestamp.

Requirement:

1. Implement a hybrid sorting system that accepts an array of Book records sorted by accession ID.
2. Your system must apply Insertion Sort for small sub-arrays (size < 10) and Quick Sort (with median-of-three pivot selection) for larger partitions.
3. Provide a benchmark function comparing the execution time and comparison count of this Hybrid Sort against pure Bubble Sort and Selection Sort for a dataset of 5,000 records.
4. Include custom structs and full memory management.

Question 2 (10 Marks): Binary Search with Range Queries

Scenario & Requirements:

Patrons frequently search for books within a specific accession ID range [min_id, max_id].

Requirement:

1. Using the sorted array from Question 1, implement a modified Binary Search algorithm that finds the first occurrence (lower bound) and last occurrence (upper bound) of books within the given range in O(log N) time.
2. Display all books in the range.
3. Write a Linear Search function and demonstrate the performance difference when handling queries on non-indexed data.

PRACTICE EXAM SET 2: COLLABORATIVE DOCUMENT EDITOR

Topics: Linked Lists (SLL, DLL), Stacks (Infix/Postfix, Undo/Redo) **Time: 1.5 Hours | Marks: 20**

Question 1 (10 Marks): Dual-List Buffer Management

Scenario & Requirements:

An advanced text editor manages paragraphs using a Doubly Linked List (DLL), where each node stores line text, line number, and character count.

Requirement:

1. Implement operations to insert a line at any index, delete a line, and swap two lines in O(1) pointer updates once nodes are located.
2. Implement a secondary Singly Linked List (SLL) that acts as a clipboard holding cut lines.
3. Write a function to paste content from the SLL clipboard into any position in the DLL text structure.

Question 2 (10 Marks): Undo/Redo Engine & Expression Evaluator

Scenario & Requirements:

The editor supports mathematical dynamic fields (e.g., inline formula evaluations like 'SUM(5, 3*2)') and an undo/redo system.

Requirement:

1. Build an Undo/Redo mechanism using two custom Stack implementations (UndoStack and RedoStack).
2. Implement an expression evaluator that converts user inline Infix formulas containing operators (+, -, *, /, ^) and nested parentheses into Postfix notation and evaluates the Postfix expression using a Stack.

PRACTICE EXAM SET 3: HOSPITAL TRIAGE & EMERGENCY OPERATIONS

Topics: Queue (Standard, Circular, Priority Queue), Sorting (Counting, Radix)

Time: 1.5 Hours | Marks: 20

Question 1 (10 Marks): Emergency Room Priority Queue & Circular Waitlist

Scenario & Requirements:

A hospital Emergency Department categorizes patients by triage severity level (1 = Critical, 5 = Non-urgent).

Requirement:

1. Implement a Priority Queue using a min-heap or sorted linked list to manage incoming patients such that emergency cases are treated immediately.
2. For non-urgent consultation rooms, implement an array-based Circular Queue to manage patients waiting for routine check-ups in a round-robin manner without array shifting.
3. Write functions for enqueue(), dequeue(), peek(), and full/empty checks for both queue types.

Question 2 (10 Marks): Linear Time Sorting for Patient Medical Records

Scenario & Requirements:

The hospital needs to organize 10,000 patient records by their 6-digit National ID and triage scores ranging from 1 to 5.

Requirement:

1. Implement Radix Sort (LSD) using Counting Sort as a stable sub-routine to sort patient IDs in O(d * (N + k)) time.
2. Write a separate function using pure Counting Sort to categorize patients by triage severity.
3. Output step-by-step state logs demonstrating how stability is preserved across passes.

PRACTICE EXAM SET 4: CLOUD FILE HIERARCHY SYSTEM

Topics: Binary Tree, General Tree, BST Functions

Time: 1.5 Hours | Marks: 20

Question 1 (10 Marks): Directory Tree & First-Child Next-Sibling Conversion

Scenario & Requirements:

A cloud storage provider represents directory structures as General Trees, where a folder can contain an arbitrary number of sub-folders and files.

Requirement:

1. Define a General Tree structure and write a function to construct a file directory.
2. Implement an algorithm to convert the General Tree into a First-Child Next-Sibling (FCNS) Binary Tree structure.

3. Write recursive functions to compute total folder size and calculate the height of the resulting Binary Tree.

Question 2 (10 Marks): Metadata Indexing using Binary Search Tree

Scenario & Requirements:

To quickly find files by unique file hash keys, the system maintains a Binary Search Tree (BST).

Requirement:

1. Implement complete BST functions: Insert(key), Search(key), and Delete(key) handling all three node deletion cases (0, 1, or 2 children).
2. Add helper functions for finding FindMin(), FindMax(), and performing Inorder, Preorder, and Postorder Traversals.
3. Write a function that returns the total count of leaf nodes and internal nodes in the BST.

PRACTICE EXAM SET 5: HIGH-FREQUENCY NETWORK ROUTING & RED-BLACK BALANCING

Topics: Red-Black Tree (Insertion, Deletion, Traversal, Searching), Advanced Tree Operations

Time: 1.5 Hours | Marks: 20

Question 1 (10 Marks): Self-Balancing IP Routing Table (RBT Insertion)

Scenario & Requirements:

A high-speed core router maintains an IP routing table keyed by 32-bit numerical IP address ranges. The tree must stay strictly balanced to guarantee O(log N) lookup latency.

Requirement:

1. Implement a Red-Black Tree (RBT) data structure supporting key insertion.
2. Handle all re-balancing cases during insertion: recoloring, Left Rotation, and Right Rotation.
3. Write a visual traversal function that outputs the tree in Level-Order format, explicitly printing each node's Key, Color (RED/BLACK), and Parent Key.

Question 2 (10 Marks): Dynamic Router Node Removal & Verification

Scenario & Requirements:

When network router nodes go offline, their keys must be dynamically removed from the Red-Black Tree without violating balancing invariants.

Requirement:

1. Implement the complete Red-Black Tree Deletion algorithm, including fix-up cases for double-black nodes.
2. Write a verification function `isValidRBTree()` that asserts the 5 core properties of Red-Black Trees:
   a) Every node is RED or BLACK.
   b) Root is BLACK.
   c) Leaves (NIL nodes) are BLACK.
   d) Red node children are BLACK (No double REDs).
   e) Every path from a node to descendant NULL nodes contains the same number of BLACK nodes.
3. Demonstrate searching for an IP range key and deleting it while proving structural integrity.